NAME:B.Sanjana

ROLL:2303A51455

BATCH:07

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Welcome | lab12.py

Users > sanjanareddy > lab12.py > merge > merged

```python
#Generate a Python function merge_sort(arr) to sort a list in ascending order. Include time and space
def merge_sort(arr):
    """
    Sorts a list in ascending order using the merge sort algorithm.

    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used for merging.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: A new sorted list in ascending order.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """Merges two sorted lists into a single sorted list.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

```

Welcome lab12.py

Users > sanjanareddy > lab12.py > merge > merged

```python
def merge(left, right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them to merged
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them to merged
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1

    return merged
# Example usage:
if __name__ == "__main__":
    unsorted_list = [38, 27, 43, 3, 9, 82, 10]
    sorted_list = merge_sort(unsorted_list)
    print("Sorted list:", sorted_list)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) sanjanareddy@Sanjanas-MacBook-Air ~ % /usr/local/bin/python3 /Users/sanjanareddy/lab1
(base) sanjanareddy@Sanjanas-MacBook-Air ~ % /usr/local/bin/python3 /Users/sanjanareddy/lab1
Sorted list: [3, 9, 10, 27, 38, 43, 82]
```

Conclusion:-

Merge Sort is efficient for large datasets because it has O(n log n) time complexity in all cases.

Task 2 – Binary Search

```python
sanjanareddy > lab12.py > ...
#Create a Python function binary_search(arr, target) that returns index or -1. Include complexity explanation.
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1
# Complexity Explanation:
# The time complexity of binary search is O(log n) because with each iteration, it effectively halves the search space. The space complexity is O(1) for the ite
# Example usage:
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
target = 5
result = binary_search(arr, target)
if result != -1:
    print(f"Element found at index: {result}")
else:    print("Element not found in the array.")
```

Task 3 – Inventory Management System

Operation	Algorithm	Justification
Search by ID	Binary Search / Hashing.	Fast O(log n) or O(1)
Sort by Price	Merge Sort	Stable and efficient
Sort by Quantity	Merge Sort	Good for large

data Justification:-

Since the dataset contains thousands of products, efficient algorithms like Binary Search and Merge Sort are suitable. Binary Search provides O(log n) time for searching. Merge Sort ensures stable sorting with O(n log n) complexity.

Task 4

HospitalManagement

Recommended Algorithms

Search by ID-

BinarySearch

Sort by Severity → Merge

Sort

Justification:-

Severity sorting requires stable sorting, so Merge Sort is preferred. Searching large patient records is efficient using Binary Search.

Task 5 – University Result System

Operation	Algorithm	Reason
Search by Roll	Binary Search.	Fast lookup
Sort by Marks	Merge Sort	Accurateranking

Welcome lab12.py

Users > sanjanareddy > lab12.py > ...

```python
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    return merge(left, right, key)
def merge(left, right, key):...
def binary_search_students(students, roll):
    left, right = 0, len(students) - 1

    while left <= right:
        mid = (left + right) // 2
        if students[mid]["roll"] == roll:
            return students[mid]
        elif students[mid]["roll"] < roll:
            left = mid + 1
        else:
            right = mid - 1
    return None
students = [
    {"roll": 1, "name": "Asha", "marks": 85},
    {"roll": 2, "name": "Rahul", "marks": 92},
    {"roll": 3, "name": "Meena", "marks": 78},
]
rank_list = merge_sort(students, "marks")
print("Rank List:", rank_list)
students_sorted_by_roll = sorted(students, key=lambda x: x["roll"])
print("Search Student:", binary_search_students(students_sorted_by_roll, 2))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) sanjanareddy@Sanjanas-MacBook-Air ~ % /usr/local/bin/python3 /Users/sanjanareddy/lab12.py
Search Student: {'roll': 2, 'name': 'Rahul', 'marks': 92}
(base) sanjanareddy@Sanjanas-MacBook-Air ~ %
```

Task 6 – Food Delivery Platform

Operation	Algorithm	Reason
Search by Order ID	Binary Search	Quick retrieval
Sot by Delivery Time	Merge Sort	Efficientforlargedata

Welcome lab12.py

Users > sanjanareddy > lab12.py > merge_sort

```python
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    return merge(left, right, key)
def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
def binary_search_orders(orders, order_id):
    left, right = 0, len(orders) - 1

    while left <= right:
        mid = (left + right) // 2
        if orders[mid]["id"] == order_id:
            return orders[mid]
        elif orders[mid]["id"] < order_id:
            left = mid + 1
        else:
            right = mid - 1
    return None
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) sanjanareddy@Sanjanas-MacBook-Air ~ % /usr/local/bin/python3 /Users/sanjanareddy/lab12.py
Search Order: {'id': 202, 'restaurant': 'KFC', 'price': 180, 'time': 20}
(base) sanjanareddy@Sanjanas-MacBook-Air ~ %
```

Conclusion:-

AI assistance helped in generating optimized and well-documented code quickly. It also helped understand complexity analysis and algorithm comparison effectively.